Тема: Створення API за допомогою Node.js та Express: Частина 2. Підключення бази даних MySQL

Мета: навчитися підключати сервер Express до бази даних MySQL та налаштувати маршрути, які відповідають методам GET, POST, PUT та DELETE, де вказуються певні шляхи та динамічні параметри маршруту. Також дізнатися, як відправляти запити HTTP до API для сервера за допомогою cURL, модуля request Node.js та HTML-форм.

Порядок виконання роботи:

1. Створено нову базу даних api, у якій створено таблицю users та заповнено її початковими тестовими даними користувачів (Humeniuk Denys, Haliuk Ivan). Для взаємодії сервера з базою даних створено підкаталог data та файл config.js, де налаштовано пул підключень за допомогою модуля mysql (рис. 1).

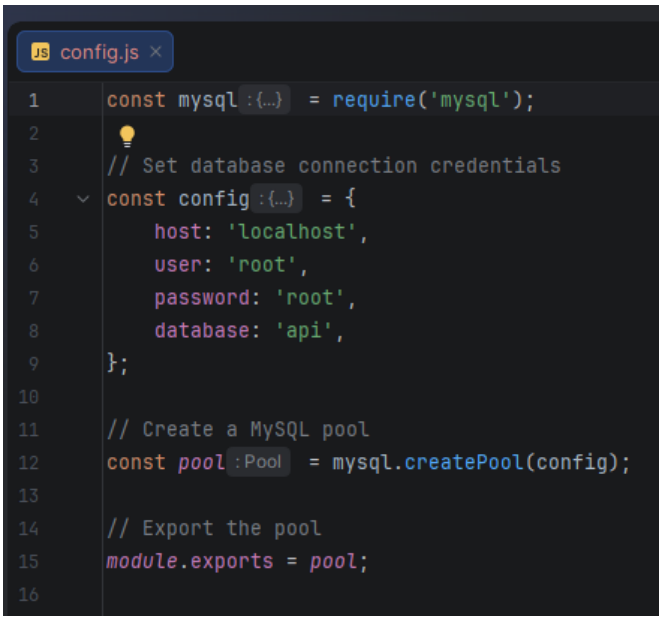


Рисунок 1 – Конфігураційний файл config.js з налаштуванням пулу підключень MySQL

2. Перед переходом до використання бази даних розглянуто початковий стан модуля routes/routes.js, де масив користувачів users створювався вручну як статичні дані (рис. 2).



Рисунок 2 – Початковий стан файлу routes.js зі статичним масивом користувачів

3. У файлі routes.js видалено статичний масив тестових даних, підключено конфігурацію пулу бази даних та оновлено обробник кінцевої точки /users за методом GET з використанням SQL-запиту SELECT * FROM users (рис. 3).

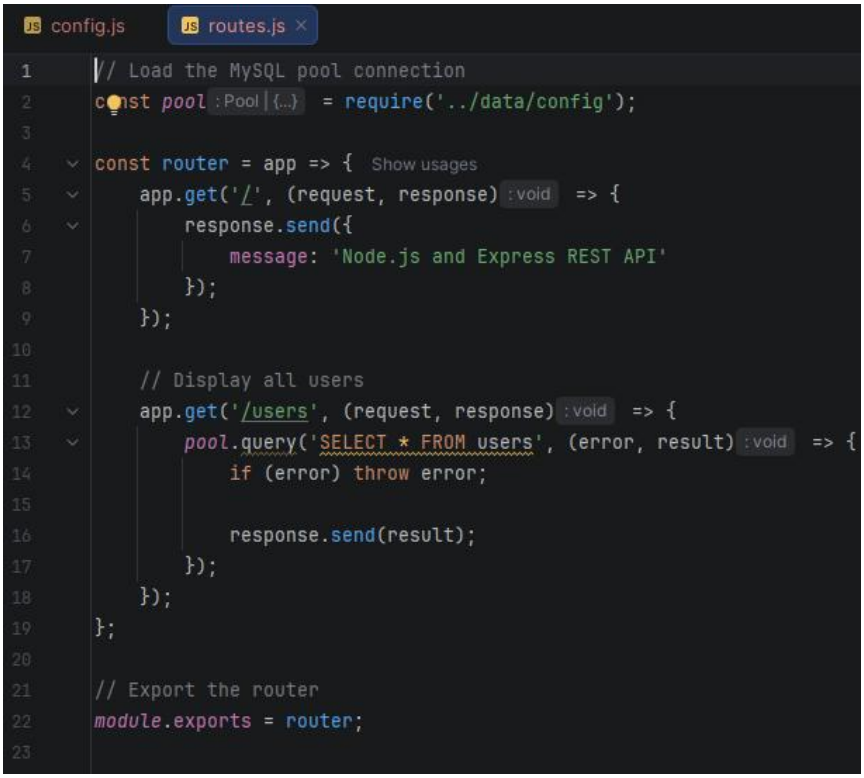


Рисунок 3 – Оновлення маршруту /users для отримання даних через запит SELECT

4. У терміналі середовища розробки WebStorm запущено Express-сервер за допомогою команди npm start, який розпочав прослуховування порту 3002 (рис. 4).

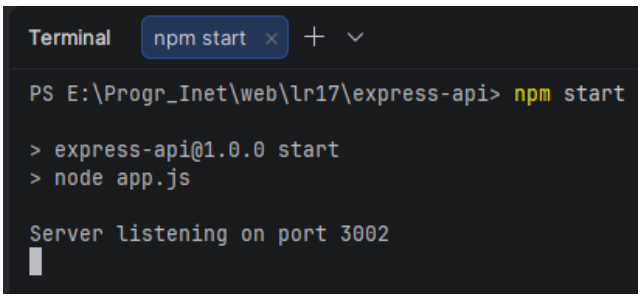


Рисунок 4 – Запуск Express-сервера у терміналі WebStorm командою npm start

5. У браузері Opera GX перевірено отримання даних з бази даних за адресою http://localhost:3002/users, де коректно відобразився масив користувачів у форматі JSON (рис. 5).

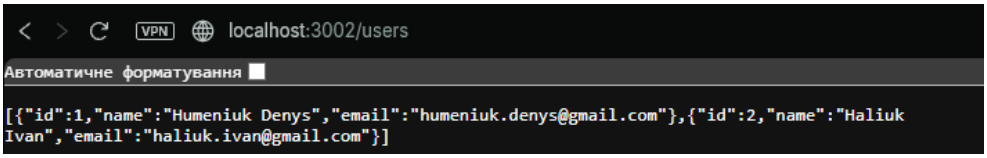


Рисунок 5 – Отримання списку користувачів з бази даних у браузері Opera GX

6. Для реалізації можливості вибірки індивідуального користувача за його ідентифікатором додано динамічний параметр маршруту /users/:id та відповідний SQL-запит з умовою WHERE id = ? (рис. 6).

```
config.js   routes.js
1   // Load the MySQL pool connection
2   const pool :Pool | {…}  = require('../data/config');
3
4   const router = app => {  Show usages
5       app.get('/', (request, response) :void  => {
6           response.send({
7               message: 'Node.js and Express REST API'
8           });
9       });
10
11      // Display all users
12      app.get('/users', (request, response) :void  => {
13          pool.query('SELECT * FROM users', (error, result) :void  => {
14              if (error) throw error;
15
16              response.send(result);
17          });
18      });
19
20      // Display a single user by ID
21      app.get('/users/:id', (request, response) :void  => {
22          const id = request.params.id;
23
24          pool.query('SELECT * FROM users WHERE id = ?', id, (error, result) :void  => {
25              if (error) throw error;
26
27              response.send(result);
28          });
29      });
30  };
31
32  // Export the router
33  module.exports = router;
34
```

Рисунок 6 – Додавання обробника маршруту /users/:id з параметром маршруту

7. Перезапущено сервер та перевірено роботу параметризованого маршруту в браузері за адресою http://localhost:3002/users/2, у результаті чого повернуто дані лише користувача з id 2 (рис. 7).

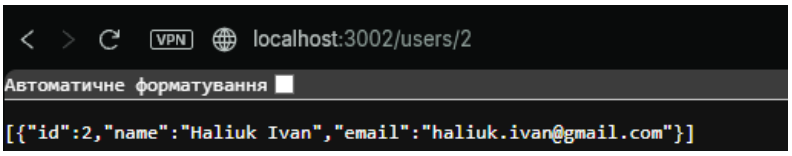


Рисунок 7 – Отримання даних користувача за його ідентифікатором у браузері

8. До модуля маршрутизації routes.js додано обробник методу POST для додавання нових користувачів за допомогою SQL-запиту INSERT INTO users SET ? з поверненням статусу 201 Created та ідентифікатора insertId (рис. 8).



Рисунок 8 – Додавання обробника запиту за методом POST до файлу routes.js

9. За допомогою утиліти cURL з прапорцем -d відправлено тестовий запит POST на адресу http://localhost:3002/users та отримано підтвердження успішного створення нового запису з id 3 (рис. 9).



Рисунок 9 – Надсилання тестового запиту POST за допомогою cURL

10. У браузері оновлено сторінку http://localhost:3002/users та перевірено наявність щойно доданого користувача у загальному списку (рис. 10).



Рисунок 10 – Відображення оновленого списку користувачів після додавання нового запису

11. Реалізовано ідемпотентний метод PUT для оновлення даних існуючого користувача за адресою /users/:id за допомогою запиту UPDATE users SET ? WHERE id = ? (рис. 11).

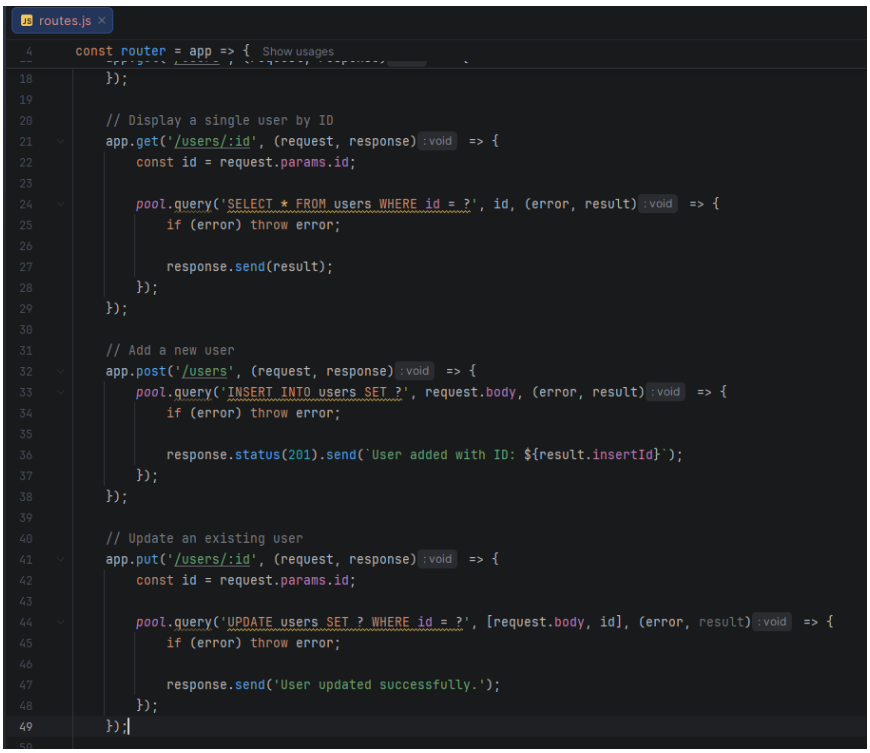


Рисунок 11 – Додавання обробника запиту за методом PUT для оновлення даних

12. Виконано запит cURL за методом PUT з прапорцем -X PUT для зміни даних користувача з id 2 на ім'я Bertram Gilfoyle та пошту bertram@piedpiper.com, у відповідь отримано повідомлення про успішне оновлення (рис. 12).



Рисунок 12 – Оновлення даних користувача за допомогою команди cURL з методом PUT

13. Перевірено відображення оновленої інформації про користувача з id 2 у браузері Opera GX (рис. 13).

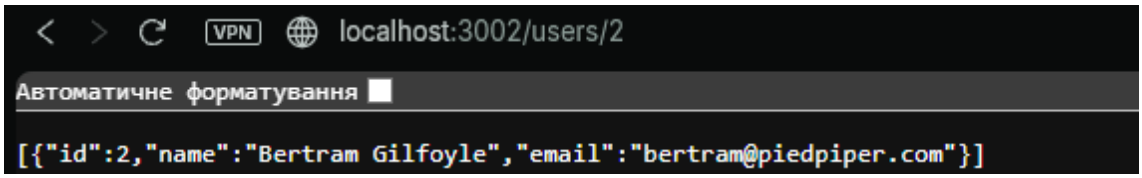


Рисунок 13 – Перевірка успішної зміни даних користувача з id 2 у браузері

14. Додано обробник запиту за методом DELETE для видалення користувача з бази даних за допомогою запиту DELETE FROM users WHERE id = ? (рис. 14).



Рисунок 14 – Додавання обробника запиту за методом DELETE для видалення користувача

15. За допомогою утиліти cURL відправлено команду видалення користувача з id 3 (curl.exe -X DELETE http://localhost:3002/users/3) та зафіксовано успішну відповідь User deleted (рис. 15).



Рисунок 15 – Надсилання запиту на видалення користувача за допомогою cURL

16. У браузері перевірено список користувачів після видалення id 3: у базі залишилися користувач 1 та оновлений користувач 2 (Bertram Gilfoyle) (рис. 16).



Рисунок 16 – Перевірка списку користувачів у браузері після видалення запису з id 3 (id 2 зберігає оновлені дані)

17. Створено файл post.js для відправлення запиту на додавання користувача за методом POST засобами бібліотеки request без використання сторонніх утиліт (рис. 17).



Рисунок 17 – Програмний код файлу post.js для надсилання запиту за допомогою модуля request

18. У терміналі WebStorm запущено скрипт node post.js: оскільки в СУБД MySQL лічильник AUTO_INCREMENT не повертається назад після видалення запису id 3, новому користувачеві призначено ідентифікатор id 4 (рис. 18).



Рисунок 18 – Запуск скрипта post.js у терміналі WebStorm з отриманням нового ідентифікатора id 4

19. У браузері зафіксовано додавання нового запису через модуль request з ідентифікатором id 4 (рис. 19).



Рисунок 19 – Перевірка списку користувачів після додавання запису через модуль request (id 4) у браузері Opera GX

20. Створено файл index.html зі стандартною HTML-формою для відправлення POST-запиту на сервер за адресою http://localhost:3002/users (рис. 20).

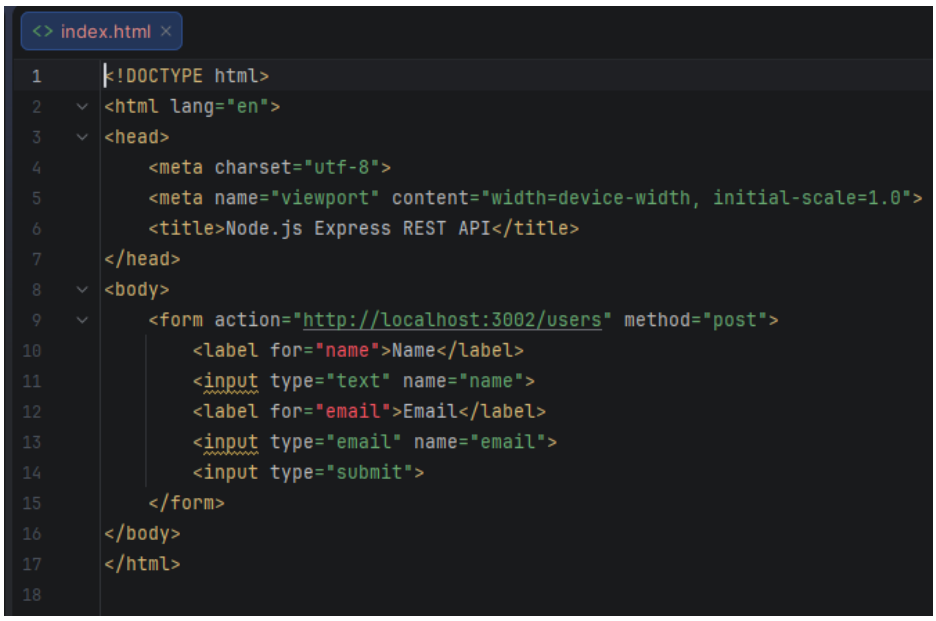


Рисунок 20 – Програмний код HTML-форми у файлі index.html

21. Відкрито створену вебсторінку index.html у браузері Opera GX та заповнено поля введення Name (Ivan Haliuk) і Email (ivan.haliuk@email.com) (рис. 21).

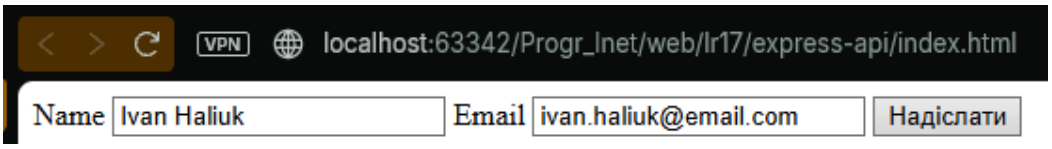


Рисунок 21 – Відображення HTML-форми у браузері Opera GX із заповненими полями

22. Після відправлення форми сервер повернув відповідь про успішне додавання нового запису з наступним автоматичним ідентифікатором id 5 (рис. 22).



Рисунок 22 – Результат обробки форми сервером із призначенням наступного ідентифікатора id 5

Індивідуальне завдання

Для книг використовуються поля id, title, author, category і price. Таблицю доповнено полями title_en та category_en для англійських назв, image для шляху до обкладинки, url і category_url для посилань, old_price для попередньої ціни, available для наявності, badge_type для позначки та display_order для порядку відображення. Поле price має тип DECIMAL(10,2), що дозволяє зберігати ціни з копійками. Наявні записи збережено, а відсутні книги попереднього каталогу додано до таблиці. Файли обкладинок зберігаються в папці images, у базі записано їхні шляхи.

GET /books повертає список книг із MySQL у визначеному порядку, GET /books/:id повертає окрему книгу. POST, PUT та DELETE відповідно додають, оновлюють і видаляють записи. Для значень у SQL-запитах використовуються заповнювачі ?.

```
    // Display all books
    app.get('/books', (request, response) => {
        pool.query('SELECT * FROM books ORDER BY display_order, id', (error,
result) => {
            if (error) throw error;

            response.send(result);
        });
    });

    // Display a single book by ID
    app.get('/books/:id', (request, response) => {
        const id = request.params.id;

        pool.query('SELECT * FROM books WHERE id = ?', id, (error, result) => {
            if (error) throw error;

            response.send(result);
        });
    });

    // Add a new book
    app.post('/books', (request, response) => {
        pool.query('INSERT INTO books SET ?', request.body, (error, result) => {
            if (error) throw error;

            response.status(201).send(`Book added with ID: ${result.insertId}`);
        });
    });

    // Update an existing book
    app.put('/books/:id', (request, response) => {
        const id = request.params.id;

        pool.query('UPDATE books SET ? WHERE id = ?', [request.body, id],
(error, result) => {
            if (error) throw error;
```

```
            response.send('Book updated successfully.');
        });
    });

    // Delete a book
    app.delete('/books/:id', (request, response) => {
        const id = request.params.id;

        pool.query('DELETE FROM books WHERE id = ?', id, (error, result) => {
            if (error) throw error;

            response.send('Book deleted.');
        });
    });
```

У catalog/scripts/common.js реалізовано спільну функцію loadProducts(). Вона отримує JSON із /books та приводить поля записів бази до структури карток каталогу. Числові значення цін перетворюються на Number. Масив PRODUCTS містить завантажені з бази товари та використовується для розрахунку корзини.

```
const PRODUCTS = {};

// Завантажує товари з MySQL через Express API для каталогу і корзини
async function loadProducts() {
    const response = await fetch("http://localhost:3002/books");
    const books = await response.json();
    const catalog = {
        products: books.map(book => ({
            id: book.id,
            name: book.title,
            nameEn: book.title_en,
            category: book.category,
            categoryEn: book.category_en,
            categoryUrl: book.category_url || "#",
            price: Number(book.price),
            oldPrice: Number(book.old_price),
            image: book.image || "images/book-placeholder.svg",
            url: book.url || "#",
            available: Boolean(book.available),
            badge: book.badge_type,
            badgeType: book.badge_type
        }))
    };

    Object.keys(PRODUCTS).forEach(id => delete PRODUCTS[id]);
    catalog.products.forEach(product => {
        PRODUCTS[product.id] = product;
    });

    return catalog;
}

function productName(product) {
    return language === "en" && product.nameEn ? product.nameEn : product.name;
}

function productCategory(product) {
    return language === "en" && product.categoryEn ? product.categoryEn :
product.category;
}
```

У catalog/scripts/catalog.js назва і категорія картки визначаються функціями productName(product) та productCategory(product), а не статичним словником книг. У catalog/scripts/cart.js назва товару також береться з productName(product), а ціна та сума розраховуються на основі даних MySQL. Якщо англійська назва нової книги не задана, відображається її основна назва. Переклади текстів інтерфейсу залишено у файлах locales/uk.json і locales/en.json.

У app.js додано роздавання сторінок каталогу та його ресурсів. Каталог відкривається за адресою http://localhost:3002/catalog/index.html, корзина – http://localhost:3002/catalog/cart.html, форма керування – http://localhost:3002/express-api/books.html. Конфігураційний файл бази даних не роздається як статичний ресурс.

```
const path = require('path');

// Сторінки каталогу та його ресурси
app.use('/catalog', express.static(path.join(__dirname, '../catalog')));
app.use('/images', express.static(path.join(__dirname, '../images')));
app.use('/locales', express.static(path.join(__dirname, '../locales')));
app.get('/style.css', (request, response) => {
    response.sendFile(path.join(__dirname, '../style.css'));
});
app.get('/express-api/books.html', (request, response) => {
    response.sendFile(path.join(__dirname, 'books.html'));
});
app.get('/express-api/index.html', (request, response) => {
    response.sendFile(path.join(__dirname, 'index.html'));
});
```

На сторінці express-api/books.html збережено форму та список записів із MySQL. Окрім основних полів, форма дозволяє вказати англійські назви, обкладинку, попередню ціну, наявність і позначку. Після натискання «Редагувати» поля заповнюються даними вибраної книги. Для додавання надсилається POST /books, для збереження змін – PUT /books/:id, для видалення – DELETE /books/:id. Нижче наведено актуальний JavaScript сторінки; незмінене оформлення не дублюється.

```
const url = 'http://localhost:3002/books';
let booksData = [];

async function getBooks() {
    try {
        const res = await fetch(url);
        booksData = await res.json();
        const tbody = document.getElementById('booksList');
        tbody.innerHTML = '';
        booksData.forEach(item => {
            const row = document.createElement('tr');
            row.innerHTML = `
                <td>${item.id}</td>
```

```
                <td>${item.title}</td>
                <td>${item.author}</td>
                <td>${item.category}</td>
                <td>${item.price} грн</td>
                <td>
                    <button class="edit-btn"
onclick="startEdit(${item.id})">Редагувати</button>
                    <button class="del-btn"
onclick="delBook(${item.id})">Видалити</button>
                </td>
            `;
            tbody.appendChild(row);
        });
    } catch (err) {
        console.error(err);
    }
}

function startEdit(id) {
    const item = booksData.find(b => b.id === id);
    if (!item) return;

    document.getElementById('editId').value = item.id;
    document.getElementById('title').value = item.title;
    document.getElementById('author').value = item.author;
    document.getElementById('category').value = item.category;
    document.getElementById('price').value = item.price;
    document.getElementById('titleEn').value = item.title_en || '';
    document.getElementById('categoryEn').value = item.category_en || '';
    document.getElementById('image').value = item.image || '';
    document.getElementById('oldPrice').value = item.old_price || '';
    document.getElementById('available').value = String(item.available);
    document.getElementById('badgeType').value = item.badge_type || '';

    document.getElementById('formTitle').innerText = 'Редагування товару (ID: '
+ item.id + ')';
    document.getElementById('submitBtn').innerText = 'Зберегти зміни';
    document.getElementById('cancelBtn').style.display = 'inline-block';
    window.scrollTo({ top: 0, behavior: 'smooth' });
}

function cancelEdit() {
    document.getElementById('bookForm').reset();
    document.getElementById('editId').value = '';
    document.getElementById('formTitle').innerText = 'Додавання товару';
    document.getElementById('submitBtn').innerText = 'Додати';
    document.getElementById('cancelBtn').style.display = 'none';
}

document.getElementById('bookForm').addEventListener('submit', async (e) => {
    e.preventDefault();
    const editId = document.getElementById('editId').value;
    const book = {
        title: document.getElementById('title').value,
        author: document.getElementById('author').value,
        category: document.getElementById('category').value,
        price: Number(document.getElementById('price').value),
        title_en: document.getElementById('titleEn').value,
        category_en: document.getElementById('categoryEn').value,
        image: document.getElementById('image').value,
        old_price: Number(document.getElementById('oldPrice').value) || null,
        available: Number(document.getElementById('available').value),
        badge_type: document.getElementById('badgeType').value
    };
```

```
    const statusEl = document.getElementById('statusMsg');

    if (editId) {
        // PUT /books/:id (оновлення)
        const res = await fetch(url + '/' + editId, {
            method: 'PUT',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(book)
        });
        const msg = await res.text();
        statusEl.innerText = msg;
        cancelEdit();
    } else {
        // POST /books (додавання)
        const res = await fetch(url, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(book)
        });
        const msg = await res.text();
        statusEl.innerText = msg;
        document.getElementById('bookForm').reset();
    }

    getBooks();
    setTimeout(() => { statusEl.innerText = ''; }, 3000);
});

async function delBook(id) {
    if (!confirm('Видалити книгу з ID: ' + id + '?')) return;
    await fetch(url + '/' + id, { method: 'DELETE' });
    getBooks();
}

getBooks();
```

Після запуску сервера відкрито каталог, що базується на лабораторній роботі №14. Картки, обкладинки, позначки та ціни відображаються за даними таблиці books (рис. 23).

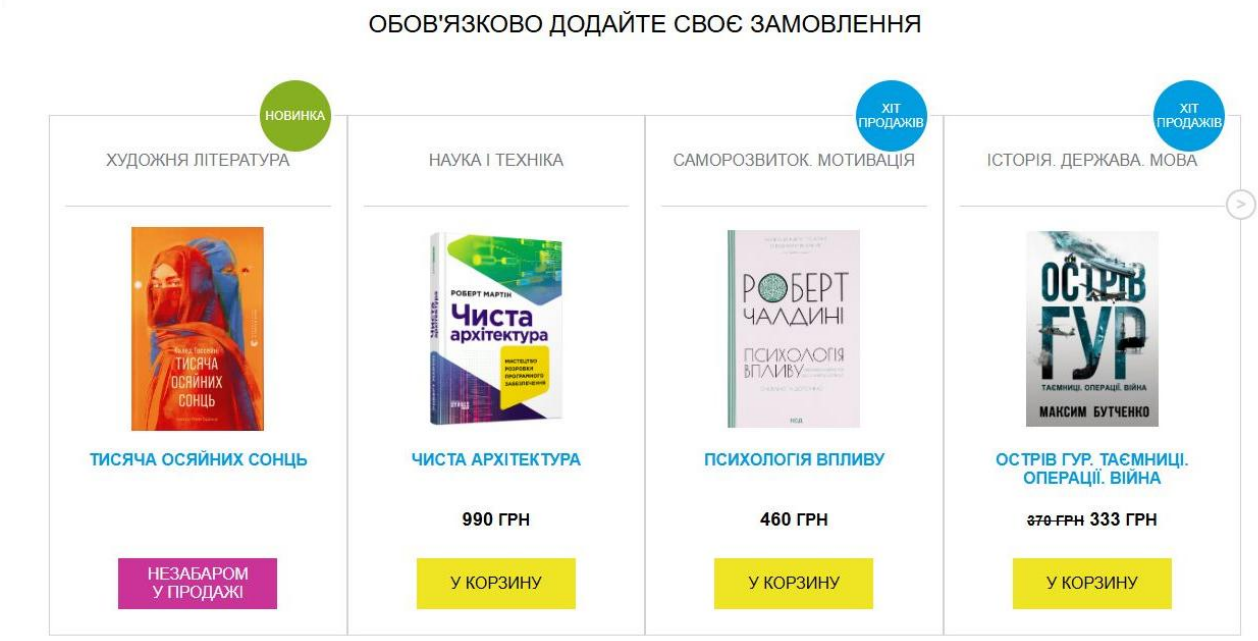


Рисунок 23 – Каталог із лабораторної роботи №14, підключений до MySQL

У формі введено дані тестової книги з початковою ціною 125,50 грн, англійською назвою Test book та шляхом до наявної обкладинки (рис. 24). Після надсилання POST-запиту новому запису призначено id 13; книга з’явилася у списку з бази даних (рис. 25).

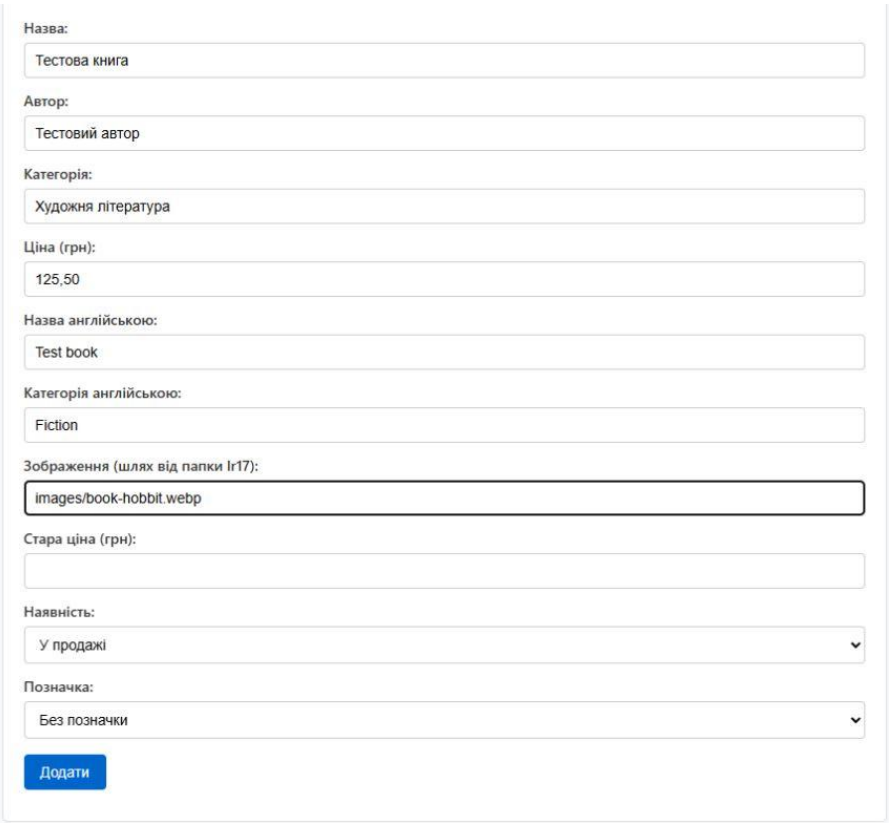
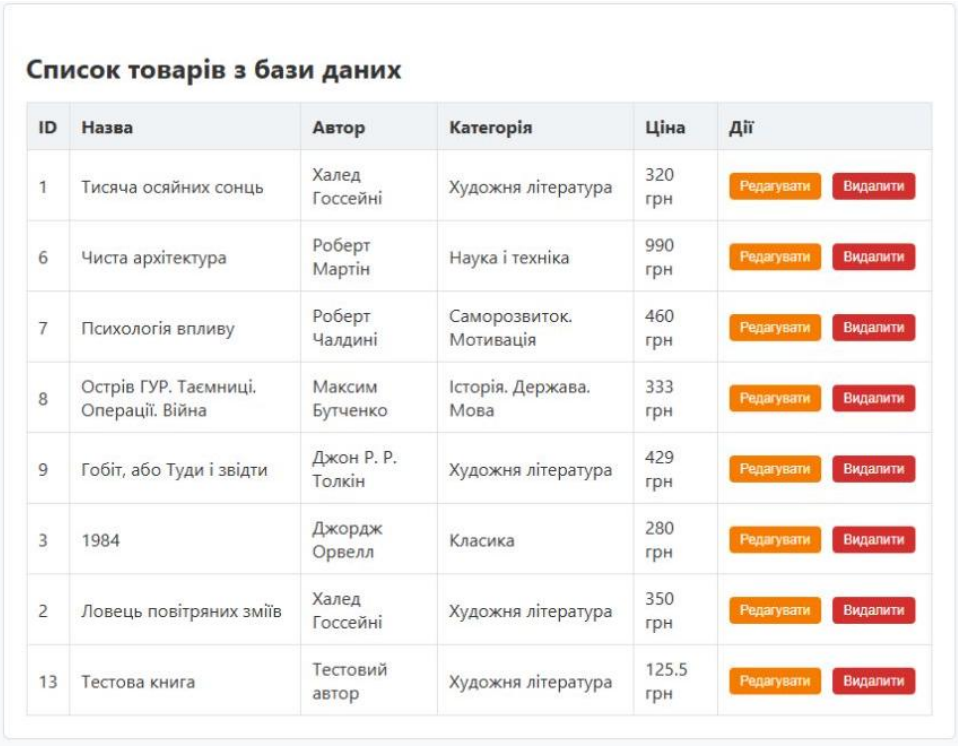


Рисунок 24 – Заповнення форми додавання тестової книги



Список товарів з бази даних

ID	Назва	Автор	Категорія	Ціна	Дії
1	Тисяча осяйних сонць	Халед Госсейні	Художня література	320 грн	Редагувати Видалити
6	Чиста архітектура	Роберт Мартін	Наука і техніка	990 грн	Редагувати Видалити
7	Психологія впливу	Роберт Чалдині	Саморозвиток. Мотивація	460 грн	Редагувати Видалити
8	Острів ГУР. Таємниці. Операції. Війна	Максим Бутченко	Історія. Держава. Мова	333 грн	Редагувати Видалити
9	Гобіт, або Туди і звідти	Джон Р. Р. Толкін	Художня література	429 грн	Редагувати Видалити
3	1984	Джордж Орвелл	Класика	280 грн	Редагувати Видалити
2	Ловець повітряних зміїв	Халед Госсейні	Художня література	350 грн	Редагувати Видалити
13	Тестова книга	Тестовий автор	Художня література	125.5 грн	Редагувати Видалити

Рисунок 25 – Додана книга з id 13 і ціною 125,50 грн у списку MySQL

Через форму редагування ціну тестової книги змінено на 199,99 грн. Після повторного відкриття каталогу та переходу стрілками до останніх карток відобразилася тестова книга з оновленою ціною (рис. 26). Це підтверджує, що сторінка каталогу отримує зміни з бази, а не з початкового статичного JSON.

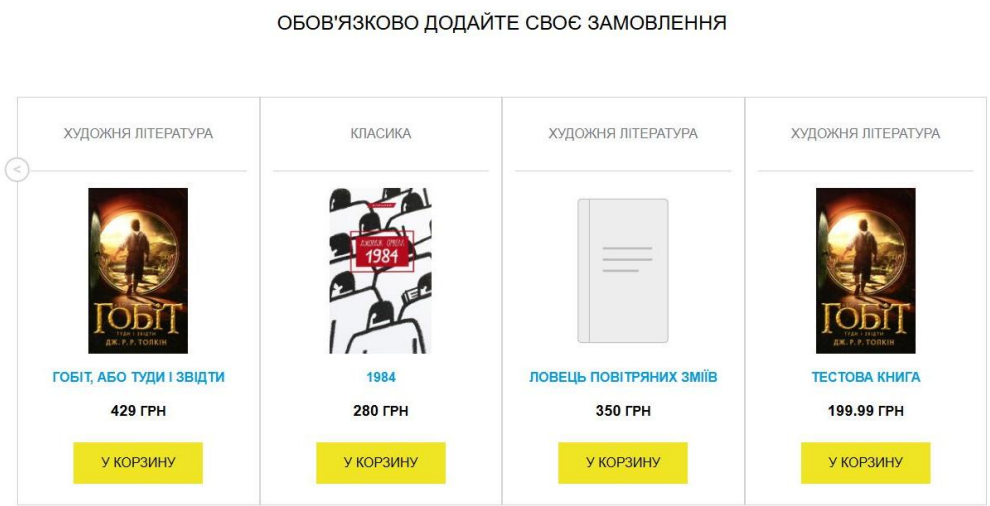


Рисунок 26 – Картка тестової книги з ціною 199,99 грн після PUT-запиту

Для завершення перевірки надіслано DELETE-запит до /books/13. Сервер повернув повідомлення Book deleted. Повторне зчитування запису безпосередньо з MySQL підтвердило його відсутність. Після оновлення сторінки тестова книга більше не відображається у списку (рис. 27).



Рисунок 27 – Список книг після видалення тестового запису з id 13

Також перевірено додавання товару до корзини, зміну кількості, розрахунок загальної суми та перемикання української й англійської мов. Окремою перевіркою CRUD підтверджено збереження записів у MySQL після перезапуску Express-сервера.

Висновки: під час лабораторної роботи навчилися підключати сервер Express до бази даних MySQL та налаштувати маршрути, які відповідають методам GET, POST, PUT та DELETE, де вказуються певні шляхи та динамічні параметри маршруту. Також дізналися, як відправляти запити HTTP до API для сервера за допомогою cURL, модуля request Node.js та HTML-форм.